

/* This note has been compiled from various sources and can be taken as reference only*/

Chapter 5 File Systems

File Naming: Structure, Type, Access, Attribute and Operation

Files are the units to store information on external storage media that satisfies the following requirements.

- a) It must store large volume of information.
- b) It must be independent i.e termination of process using the file should not affect the existence of information within it.
- c) Multiple processes must be able to access information within it at the same instant of time.

The part of OS dealing with files, its structure, naming, implementation, security and so on is called file system

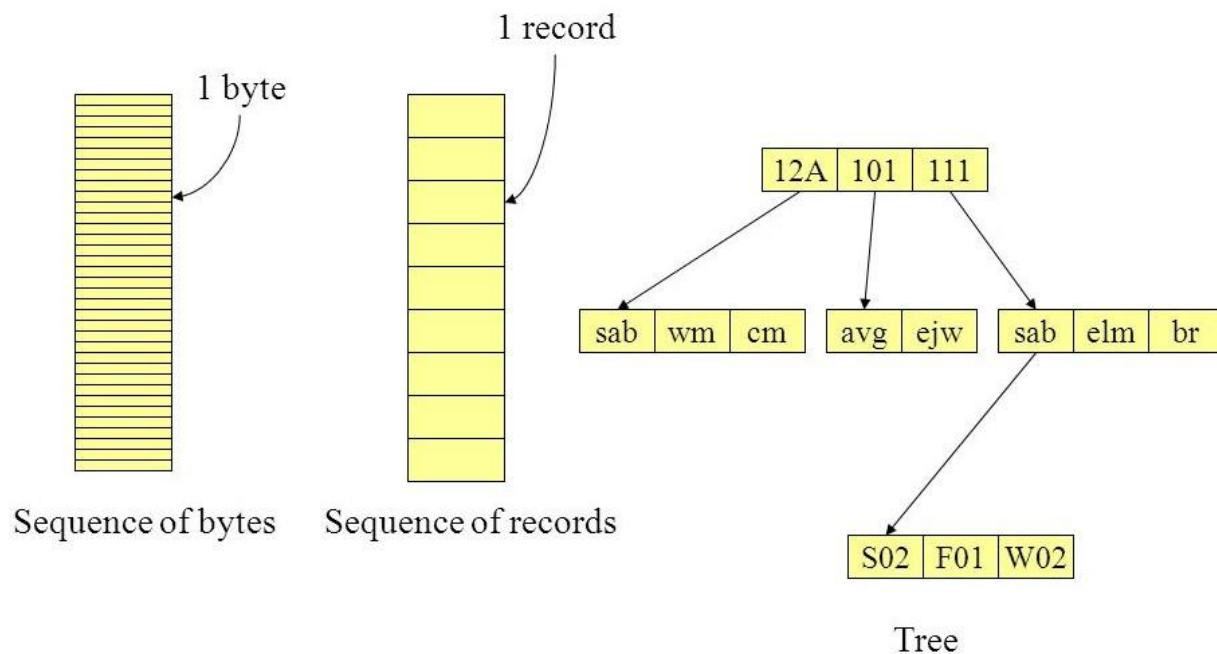
File Naming

- On creation of a file by any process, it provides a name.
 - When process terminates, the file continues to exist.
 - The rules for file naming varies from system to system.
 - Generally, files are named with two parts separated by a period sign: filename and extension.
 - Extension indicates some information about the file
 - For eg: file. c
- Here, the filename is 'file' and extension is 'c'. The extension indicates that the file is a C file which can be compiled by C compiler.

File Structure

- File can be structured as **byte sequence, record sequence and tree.**
- In byte sequence, file is an unstructured sequence of bytes. The OS only sees bytes and the meaning should be provided by user level programs.
- In record sequence, a file is a sequence of fixed length records, each with same internal structure.
- In tree a file consists of a tree of records, may be of varying length, each

containing key field in fixed position in the record. The key field is used for sorting tree.



File Types

File types refer to classifying the content of the *file*, such as a program, text *file*. There are three basic types of files recognized by the system and they are:

1. **Regular files**
2. **Directory files**
3. **Special files**

- 1. **Regular files** contain user information and data bytes.
- Regular files may be **ASCII/ text file or binary file**.
- ASCII files consist of lines of text, each line terminated by carriage return or line feed. Each line may have variable length. These files are easy to edit and display.
- **Binary files** usually have some internal structure with sections: header, text, data, relocation bits and symbol table for executable file, and header, object module for archive file.

2. Directory files

The Files those are Stored into the Particular Directory or Folder. For Example a Folder Name Songs which Contains Many Songs So that all the Files of Songs are known as Directory Files.

3. Special files

The Special Files are those which are not created by the user. Or The Files those are necessary to run a System. The Files those are created by the System

File Access

File Access Methods

When a file is used then the stored information in the file must be accessed and read into the memory of a computer system. Various mechanisms are provided to access a file from the operating system.

1. Sequential access
2. Direct Access
3. Index Access

1. Sequential Access:

It is the simplest access mechanism, in which information's stored in a file are accessed in an order such that one record is processed after the other. For example editors and compilers usually access files in this manner.

2. Direct Access:

It is an alternative method for accessing a file, which is based on the disk model of a file, since disk allows random access to any block or record of a file. For this method, a file is viewed as a numbered sequence of blocks or records which are read / written in an arbitrary manner, i.e. there is no restriction on the order of reading or writing. It is well suited for Database management System.

3. Index Access:

In this method an index is created which contains a key field and pointers to the various blocks. To find an entry in the file for a key value, we first search the index and then use the pointer to directly access a file and find the desired entry. With large files, the index file itself may become too large to be keeping in memory. One solution is to create an index for the index file. The primary index file would contain pointers to secondary index files, which would point to the actual data items

File Attributes

- The additional information except filename and its data associated with each file is known as file attributes.
- The attributes varies from system to system.
- Protection, password, creator, owner, flags (hidden, Read only, archive, ASCII / binary, random access, lock), record length, key length, creation time, Time of last access, current size, maximum size, etc.

File Operations

- Different operation are used to allow storage and retrieval of information.
- CREATE, DELETE, OPEN, CLOSE, READ, WRITE, APPEND, SEEK, GET ATTRIBUTES, SET ATTRIBUTES, RENAME.

Directory and File Paths

- Directory is the structure in file system that keeps track of the files.
- A directory contains a number of entries, one per file

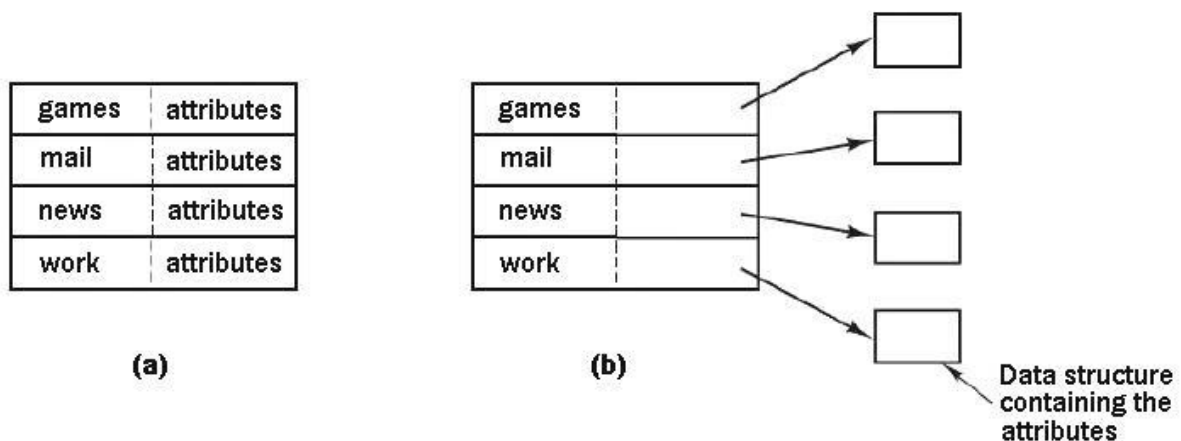


Figure 1. (a) A simple directory containing fixed-size entries with the disk addresses and attributes in the directory entry. (b) A directory in which each entry just refers to an i-node.

- The number of directories varies from system to system.
- When file system is organized as a directory tree, a way is needed to specify file names. The way is by specifying path name for each file.
- Each file is given an **absolute path** name consisting of the path from root

directory to the file

Eg: /user/ ast/ mailbox

- If first character of path name is separator, It is absolute,
- **Relative path** name is the path for the file beginning from the current working directory
- Eg: if current directory is /user/ast, then relative path can be mailbox.

- The operations allowed for managing directories are: -

- a) CREATE
- b) DELETE
- c) OPENDIR
- d) CLOSEDIR
- e) READIR
- f) RENAME
- g) LINK
- h) UNLINK

- Linking is the technique that allows a file to appear in more than one directory.

In Linux, everything is treated as a file even if it is a normal file, a directory, or even a device such as a printer or keyboard. All the directories and files are stored under one root directory which is represented by a forward slash /.

The Linux directory layout follows the **Filesystem Hierarchy Standard (FHS)**. This standard defines how directories are organized and what types of files should be stored in each. Since Linux is based on UNIX, it inherits many of its filesystem conventions. Similar directory structures can also be found in other UNIX-like operating systems such as BSD and macOS. Understanding the Linux directory structure is essential for efficient system administration, file management, and problem-solving.

Types of files in the Linux system

Linux categorizes files into different types based on their purpose, such as general data storage, system structure, or device representation.

1. **General Files** - It is also called ordinary files. It may be an image, video, program, or simple text file. These types of files can be in ASCII or Binary format. It is the most commonly used file in the Linux system.
2. **Directory Files** - These types of files are a warehouse for other file types. It may be a directory file within a directory (subdirectory).
3. **Device Files** - In a Windows-like operating system, devices like CD-ROM and hard drives are represented as drive letters like F: G: H,

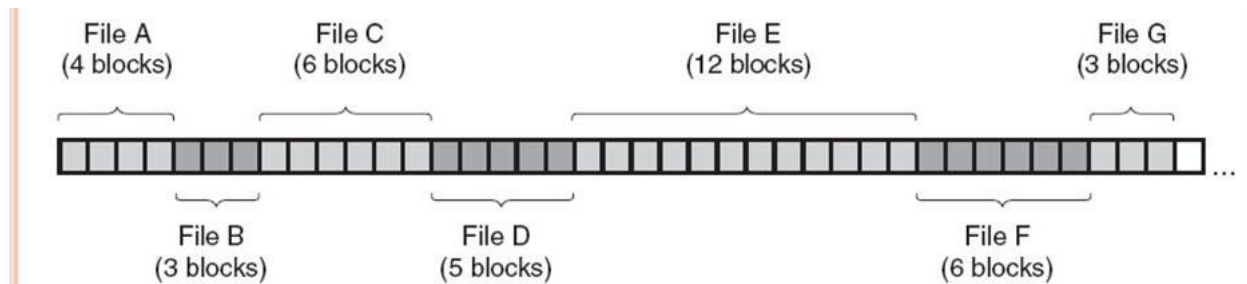
whereas in the Linux system, devices are represented as files. As for example, /dev/sda1, /dev/sda2, and so on.

File Implementation (File Allocation Technique)

It is the mechanism of keeping track of which disk blocks go which files

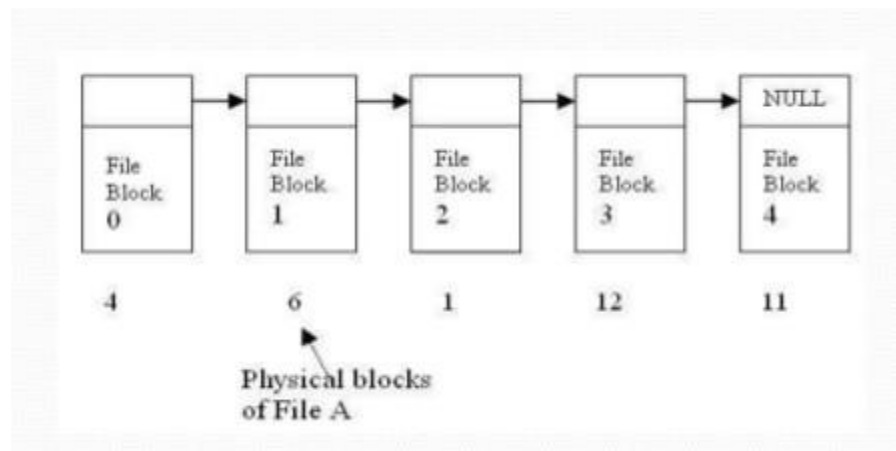
Contiguous Allocation

- Store each file as a contiguous block of data on the disk.
- Disk address is assigned in liner order.
- It is easy to implement.
- The entire file can be read from disk in a single operation.
- It is not feasible if maximum file size is unknown during creation
- It may result in disk fragmentation.



Linked Listed Allocation

- Store each file as a linked list of disk blocks.
- The first word of each block is used as a pointer to next one and the rest of the block is for data.
- Space is not lost to disk fragmentation except for internal fragmentation.
- The directory entry can store disk address of first block only.
- Random access is very slow.
- Extra overhead while copying due to a pointer at the head of block.



Linked List Allocation Using on Index

- File allocation table (FAT) keeps track of pointers as a table in memory.
- A table has one entry for each disk block with all file blocks linked together.
- The directory, stores the first block number of each file.
- The entire block is available for data.
- Random access is easier.
- The entire table must be in memory all the time.
- Growth of table size is liner with grow of disk size.

Physical Block	Pointers
0	
1	0
2	14
3	
4	12
5	
6	9
7	
8	0
9	4
10	
11	2
12	1
13	
14	8

← Unused

← File A starts here

← File B starts here

I – nodes

I-nodes

It is one of the modern method for keeping tracks of each block belongs to which file is to associate with each file within table, called i-node table. I-node is a unique number given to a file in an operating system i.e. it is a data structure that stores all the information about a file except its name and its actual data. When a file is created both name and an I node number is assigned to it, which is unique within the file system. Both name and their corresponding I node numbers are stored as entries in their directory that appears to the user to contain the file i.e. the directory associates file names and i-nodes.

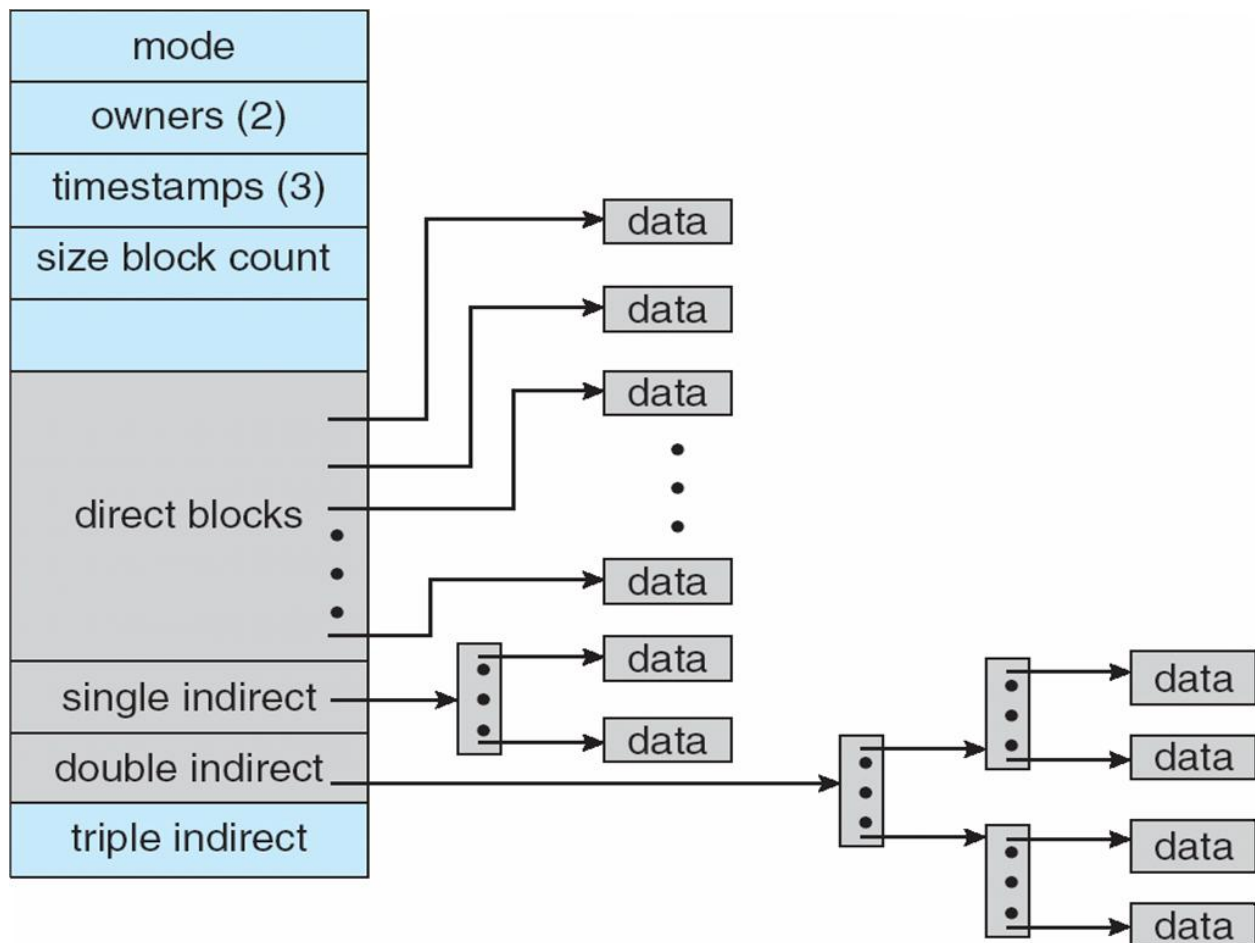
Whenever a user or a program refers to a file by name, the operating system uses that name to look up the corresponding I node, which then enables the system to obtain the information it needs about the file to perform further operations.

A i-node contains following metadata:

- ❑ Size of file (in bytes) and its physical location
- ❑ The file's owner and group
- ❑ The file's access permissions (read, write, execute)
- ❑ Timestamps telling when the i-node was created, last modified and last accessed and
- ❑ A reference count telling how many hard links points to the i-node.

- Each file is associated with a table called index-nodes (i-node) which lists the attributes and disk addresses of the file's blocks.

- Directory contains addresses of index blocks of files.
- Keep data structure in memory only for active files.
- Solves the growth problem.
- Each i-node is of fixed size.



Some common types of file systems include:

1. FAT32:

- Originally designed for broader compatibility across different operating systems and devices.
- High compatibility with various systems (Windows, macOS, Linux, gaming consoles, etc.).
- Has a maximum file size of 4GB and a maximum partition size of 8TB. It lacks advanced features like file permissions and encryption.

2. NTFS:

- The primary file system for modern Windows operating systems, offering robust features.
- Supports large files and partitions, provides file permissions, encryption, and journaling for data integrity.
- While compatible with other operating systems, it's not the default or most performant choice.

3. NFS:

- A network file system protocol that allows multiple computers to share files and directories over a network.
- Enables centralized storage and access to files from different locations on a network.
- Requires a network connection to access files; security can be a concern if not properly configured.

4. EXT4:

- The default file system for many Linux distributions, known for its performance and reliability.
- Features like journaling improve data integrity and performance, especially on larger storage devices.
- Not directly readable or writable by Windows without third-party tools.

Performance Factors

1. Fragmentation
Less Fragmentation More performance (Seek Time, read -Write time)
2. Access Time
Less access time More Performance

End of chapter 5